

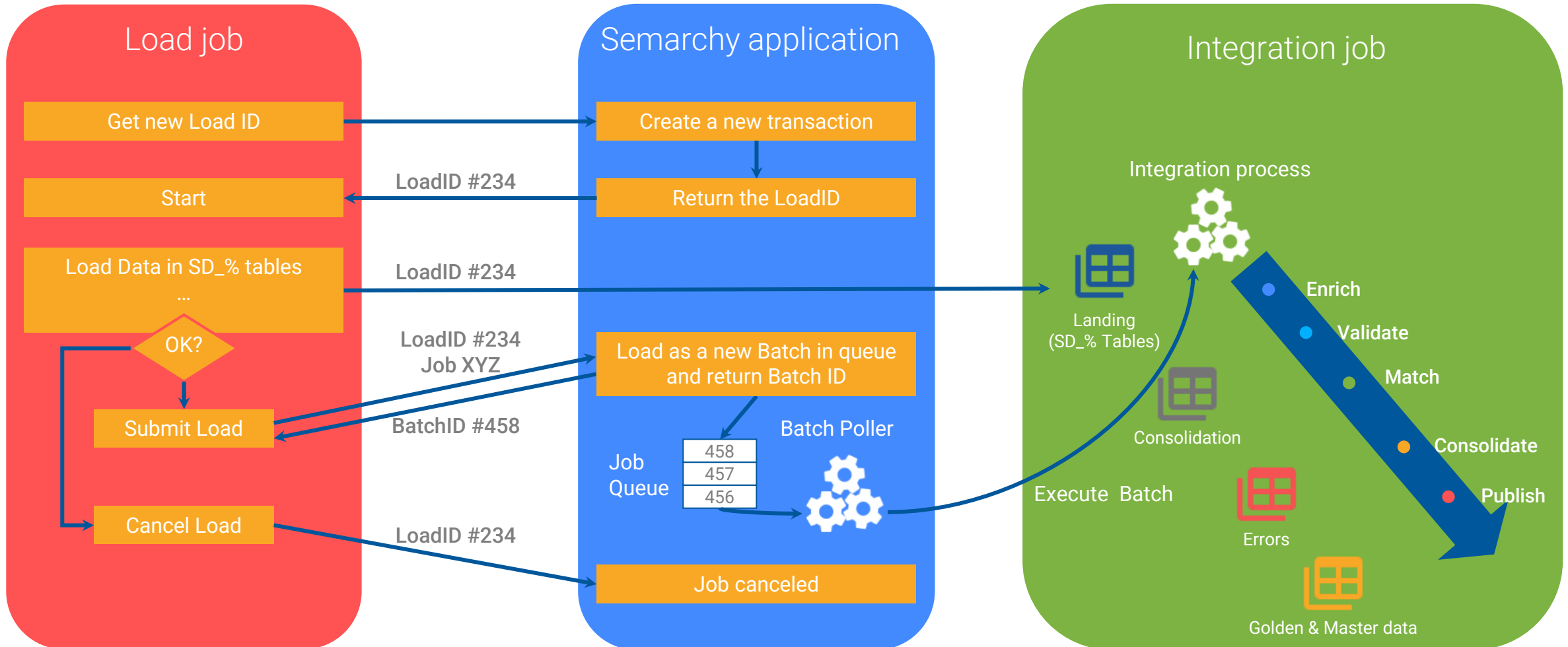


Integrating Data

Overview

```
ace:!0,async:!1,global:!1,"throws":!0}}},n
re(this[0]),b.map(function(){var
is).wrapInner(a.call(this,b))}:this.each(f
function(){return this.parent().each(function
idden"===a.type)return!0;a=a.parentNode}ret
urn!n.expr.filters.hidden(a)};var Zb=%20/g,
"object"===typeof e&&null!=e?b:"")+"]",e,c,d)
)+"="+encodeURIComponent(b)};if(void
return d.join("&").replace(Zb,"+")),n.fn.exte
a=this.type;return this.name&&!
on(a){return{name:b.name,value:a.replace(_b,
igger -> action -> reward -> investment)$/i.
jax=!!fc,fc&&n.ajaxTransport(function(b){if(
rideMimeType&&g.overrideMimeType(b.mimeType)
lf(c&&(d||4===g.readyState))if(delete ec[h],
isLocal||b.crossDomain?1223===f&&(f=204):f=
new a.XMLHttpRequest}catch(b){})function hc
vascript, application/ecmascript, applicati
Domain&&(a.type="GET",a.global=!1)),n.ajaxT
b.url,b.onload=b.onreadystatechange=fu
```

Load Process Overview





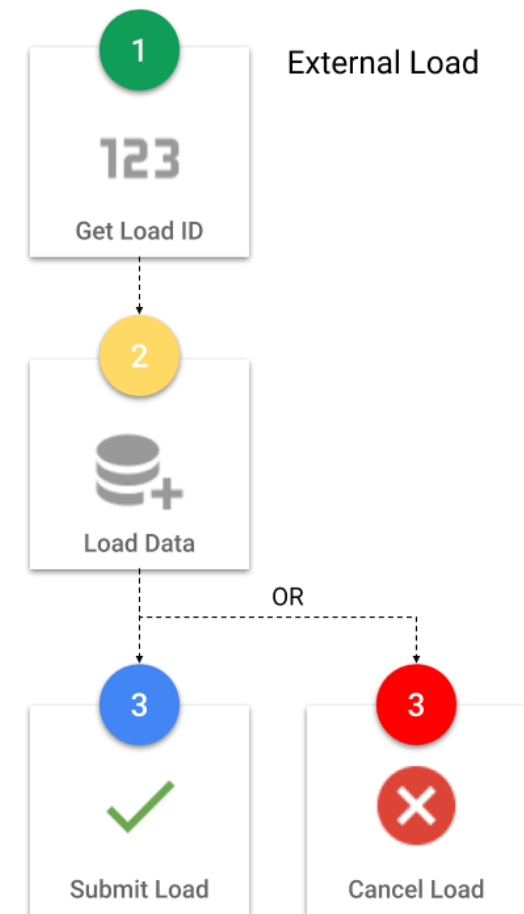
Integrating Data

Load Process

```
ace:!0,async:!1,global:!1,"throws":!0}}},n
re(this[0]),b.map(function(){var
is).wrapInner(a.call(this,b))}:this.each(f
function(){return this.parent().each(function
idden"===a.type)return!0;a=a.parentNode}ret
urn!n.expr.filters.hidden(a)};var Zb=/%20/g,
"object"===typeof e&&null!=e?b:"")+"]",e,c,d)
)+"="+encodeURIComponent(b)};if(void
return d.join("&").replace(Zb,"+")),n.fn.exte
a=this.type;return this.name&&!
on(a){return{name:b.name,value:a.replace(_b,
igger -> action -> reward -> investment)$/i.
jax=!!fc,fc&&n.ajaxTransport(function(b){if(
rideMimeType&&g.overrideMimeType(b.mimeType)
lf(c&&(d||4===g.readyState))if(delete ec[h],
isLocal||b.crossDomain?1223===f&&(f=204):f=
new a.XMLHttpRequest}catch(b){})function hc
vascript, application/ecmascript, applicati
Domain&&(a.type="GET",a.global=!1)),n.ajax
b.url,b.onload=b.onreadystatechange=fu
```

Run-time – Overview

- **Loads** are created by the ETL in 3 phases
 1. **Initialize**: Request a **load ID** identifying the load. The load is performed on a specified **data location**
 2. **Load**: Inserts data in the interface tables (SD_%). Use the **Load ID** to populate the field **B_LOADID**
 3. **Submit**: Submit the load (**load ID**) and specify an **integration job**. The platform creates a batch and returns a **batch ID** to the ETL. Jobs are processed in their order of submission
- The **integration batch poller** polls for submitted jobs and starts their processing in the engine.
- Run-time is managed in the **data locations** and/or the **Management** perspective





Integrating Data

Initialize a Load

```
ace:!0,async:!1,global:!1,"throws":!0}}},n
re(this[0]),b.map(function(){var
is).wrapInner(a.call(this,b))}:this.each(f
function(){return this.parent().each(function
idden"===a.type)return!0;a=a.parentNode}ret
urn!n.expr.filters.hidden(a)};var Zb=%20/g,
"object"===typeof e&&null!=e?b:"")+"]",e,c,d)
)+"="+encodeURIComponent(b)};if(void
return d.join("&").replace(Zb,"+")),n.fn.exte
a=this.type;return this.name&&!
on(a){return{name:b.name,value:a.replace(_b,
igger -> action -> reward -> investment)$/i.
jax=!!fc,fc&&n.ajaxTransport(function(b){if(
rideMimeType&&g.overrideMimeType(b.mimeType)
lf(c&&(d||4===g.readyState))if(delete ec[h],
isLocal||b.crossDomain?1223===f&&(f=204):f=
new a.XMLHttpRequest}catch(b){})function hc
vascript, application/ecmascript, applicati
Domain&&(a.type="GET",a.global=!1)),n.ajax
b.url,b.onload=b.onreadystatechange=fu
```

Step 1 - Initialize a Load

This process consists of requesting a **load ID** from the **Semarchy Repository schema**

```
vLoad_id := <semarchy_repository_schema>.GET_NEW_LOADID(  
    '<data_location_name>',    /* Data Location Name (As in the UI) */  
    '<program_name>',          /* Identifies the Middleware*/  
    '<load_description>',      /* Informational. Describes the load. */  
    '<user_name>'              /* User initializing the load. */  
);
```



Integrating Data

Loading SD tables

```
ace:!0,async:!1,global:!1,"throws":!0}}},n
re(this[0]),b.map(function(){var
is).wrapInner(a.call(this,b))}:this.each(f
function(){return this.parent().each(function
idden"===a.type)return!0;a=a.parentNode}ret
urn!n.expr.filters.hidden(a)};var Zb=/%20/g,
"object"===typeof e&&null!=e?b:"")+"]",e,c,d)
)+"="+encodeURIComponent(b)};if(void
return d.join("&").replace(Zb,"+")),n.fn.exte
a=this.type;return this.name&&!
on(a){return{name:b.name,value:a.replace(_b,
igger -> action -> reward -> investment)$/i.
jax=!!fc,fc&&n.ajaxTransport(function(b){if(
rideMimeType&&g.overrideMimeType(b.mimeType)
lf(c&&(d||4===g.readyState))if(delete ec[h],
isLocal||b.crossDomain?1223===f&&(f=204):f=
new a.XMLHttpRequest}catch(b){})function hc
vascript, application/ecmascript, applicati
Domain&&(a.type="GET",a.global=!1)),n.ajax
b.url,b.onload=b.onreadystatechange=fu
```

Step 2 - Loading the SD tables

- Use SQL or your regular ETL/Integration Platform
- There is no required order to load the SD_ tables
 - No FK are implemented. Validations check the references
- Make sure you load tables that are included in the job that will be submitted
 - Tables not processed by the job will be ignored
- You should preferably use a transaction
 - You can Rollback changes on the SD_ tables at will
 - `cancelLoad()` or `INTEGRATION_LOAD.CANCEL_LOAD()` or cancels the load but does not delete records from these tables

Which SD columns are mandatory?

- **B_LOADID**: Load ID provided by *getNewLoadID()* or *INTEGRATION_LOAD.GET_NEW_LOADID()*
- **B_CLASSNAME**: Class that you want to load
 - Use the entity name as it is defined in the data model
- **Primary Key**
 - Basic entities and ID-matched entities: Load the **PK column**. (Name defined in your model)
 - Fuzzy-matched entities:
 - **B_SOURCEID** should contain the primary key in the source system.
 - **B_PUBID**: Publisher ID
 - Should be a known **publisher code** defined in your model

Which SD Columns should be loaded?

- Mandatory Attributes
 - Pre-consolidation validation will reject source records with null values
- References
 - Reference to Basic Entities and ID Matching Entities
 - F_<**Physical Name of the Reference To Role Name**>
 - This ID is common to all systems, and should contain the referenced record ID
 - References to Fuzzy Matching Entities
 - FS_<**Physical Name of the Referenced To Role Name**>: Referenced record ID
 - FP_<**Physical Name of the Referenced To Role Name**>: Referenced record Publisher
 - E.g.: *FP_CUSTOMER = 'CRM', FS_CUSTOMER = '0245646'*

Which SD columns are optional?

- Other columns
 - Depending on the **validations**, certain columns need to be loaded with specific values:
 - e.g. LOV range of values
- Optional: B_CREATOR, B_UPDATOR
 - Can be optionally loaded. Defaults to the user specified in the GET_NEW_LOADID() or the connected user (WS).
- Optional: B_CREDATE, B_UPDDATE
 - Can be optionally loaded. Defaults to the current timestamp

Which SD columns to **AVOID** loading

These will be internally set by the process:

- B_ERROR_STATUS
- B_ORIGINALBATCHID
- B_OPERATIONID

Load data in SA table to manage deletes

- You can load data in SA table in order to manage deletes
 - The integration process will manage the delete as configured in the model
 - Restrict
 - Nullify
 - Cascade
- You have to load the following columns :
 - B_LOADID
 - ID of the row to be deleted
 - B_DELETETYPE (SOFT_DELETED or HARD_DELETED)
 - B_DELETEDATE
 - B_DELETEAUTHOR
 - B_DELETEOPERATION : An ID that represent the delete operation (GUID or LOADID as an example)



Integrating Data

Submit the Load

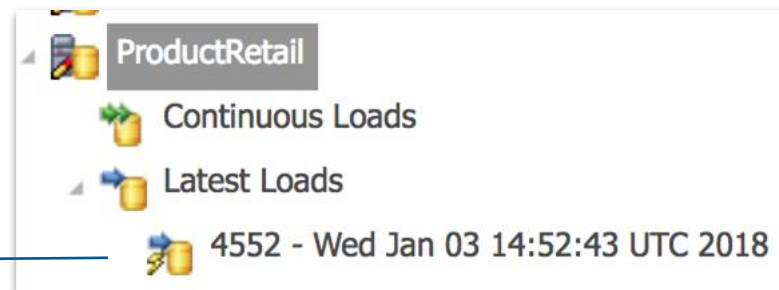
```
ace:!0,async:!1,global:!1,"throws":!0}}},n
re(this[0]),b.map(function(){var
is).wrapInner(a.call(this,b))}:this.each(f
function(){return this.parent().each(function
idden"===a.type)return!0;a=a.parentNode}ret
urn!n.expr.filters.hidden(a)};var Zb=/%20/g,
"object"===typeof e&&null!=e?b:"")+"]",e,c,d)
)+"="+encodeURIComponent(b)};if(void
return d.join("&").replace(Zb,"+")),n.fn.exte
a=this.type;return this.name&&!
on(a){return{name:b.name,value:a.replace(_b,
igger -> action -> reward -> investment)$/i.
jax=!!fc,fc&&n.ajaxTransport(function(b){if(
rideMimeType&&g.overrideMimeType(b.mimeType)
lf(c&&(d||4===g.readyState))if(delete ec[h],
isLocal||b.crossDomain?1223===f&&(f=204):f=
new a.XMLHttpRequest}catch(b){})function hc
vascript, application/ecmascript, applicati
Domain&&(a.type="GET",a.global=!1)),n.ajax
b.url,b.onload=b.onreadystatechange=fu
```

Step 3 - Submit the Load

- Loads in progress appear as running
- This process finalizes the load and submits it to the job queue

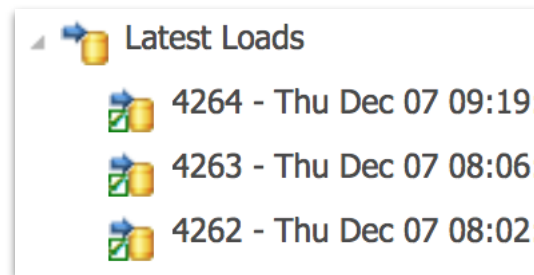
```
vBatch_id := <semarchy_repository_schema>.INTEGRATION_LOAD.SUBMIT_LOAD(  
    <load_id>,                                /* Load ID */  
    '<integration_job>',                      /* Name of the Integration Job to trigger. */  
    '<user_name>'                             /* User who has initialized the load. */  
);
```

Running



What happens after a Submit?

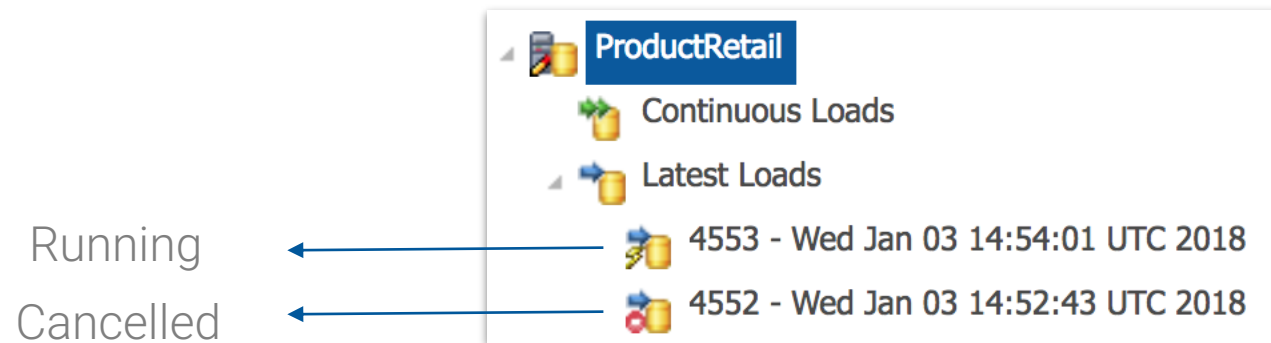
- Loads are converted into **batches**
- The **Integration Batch Poller** dispatches the jobs into their queues
 - Jobs are **processed** in their **submit order**
 - Jobs are executed and certify the golden data from the load.
- Job execution can be reviewed in the job logs
- Completed loads appear as finished



Cancel a Load

Loads in progress appear as running.

This process cancels the load, but does not delete records inserted in the SD tables





Integrating data

Integration Jobs

```
ace:!0,async:!1,global:!1,"throws":!0}}},n
re(this[0]),b.map(function(){var
is).wrapInner(a.call(this,b))}:this.each(f
function(){return this.parent().each(function
idden"===a.type)return!0;a=a.parentNode}ret
urn!n.expr.filters.hidden(a)};var Zb=/%20/g,
"object"===typeof e&&null!=e?b:"")+"]",e,c,d)
)+"="+encodeURIComponent(b)};if(void
return d.join("&").replace(Zb,"+")),n.fn.exte
a=this.type;return this.name&&!
on(a){return{name:b.name,value:a.replace(_b,
igger -> action -> reward -> investment)$/i.
jax=!!fc,fc&&n.ajaxTransport(function(b){if(
rideMimeType&&g.overrideMimeType(b.mimeType)
lf(c&&(d||4===g.readyState))if(delete ec[h],
isLocal||b.crossDomain?1223===f&&(f=204):f=
new a.XMLHttpRequest}catch(b){})function hc
vascript, application/ecmascript, applicati
Domain&&(a.type="GET",a.global=!1)),n.ajaxT
b.url,b.onload=b.onreadystatechange=fu
```

Integration Jobs

Run the data certification process and can be triggered by:

- **Data integration** (ETL, ESB, etc): SQL or REST API
 - Using **on-demand** or **Continuous** Loads
- **User interactions**: steppers, workflows, duplicate managers

 INTEGRATE_CUSTOMER360 ▼

Tasks

	Position	Name	Entity	Enable Source Enri...	Enable Source Validation	Enable Match	Enable ...	Enable
	1	Company	Company	☒	☒	☒	☒	☒
	2	FinancialReport	FinancialReport	☒	☒	☒	☒	☒
	4	Invoice	Invoice	☒	☒	☒	☒	☒
	5	Opportunity	Opportunity	☒	☒	☒	☒	☒
	6	Person	Person	☒	☒	☒	☒	☒
	7	SupportCase	SupportCase	☒	☒	☒	☒	☒

Integration Jobs

Job: INTEGRATE_CUSTOMER360

Customer360 ▾ INTEGRATE_CUSTOMER360 ▾

Details
Tasks
Job Parameters

Name	Value
PARAM_ANALYZE_STATS	1

Parameter	Value	Description
PARAM_ANALYZE_STATS	1 0	Causes the job to trigger analyze stats after each table data change
PARAM_RECYCLE_ERRORS	1 0	Causes the job to recycle previous errors into current load
PARAM_AGGREGATE_JOB_ENRICHERS	1 0	Causes the job generation to aggregate all SemQL enrichers into a single enricher
PARAM_AGGREGATE_ENTITY_ENRICHERS_{0}	1 0	Causes the job generation to aggregate all SemQL enrichers of entity {0} into a single enricher



Integrating Data

Configuring the Integration Batch Poller



Configuring the Integration Batch Poller

The screenshot displays the Semarchy xDM Application Builder interface. On the left, a navigation pane under 'Management' shows a tree structure with 'Data Locations', 'Models Administration', and 'Job Executions'. The 'Integration Batch Poller' is highlighted under 'Job Executions', marked with a blue circle and the number '1'. The main panel shows the 'Integration Batch Poller' configuration. A blue circle with the number '2' is placed over the configuration area. The configuration is divided into 'Configuration' and 'Advanced' sections. The 'Configuration' section includes radio buttons for scheduling: 'Weekly Every: Sunday' (with a time field 'at: 00 : 00'), 'Daily at: 00 : 00', 'Hourly at: 00 : 00 : 00', 'Interval in Seconds: 10' (selected), and 'Cron Syntax: 0 0 * * * ?'. The 'Advanced' section is currently collapsed. In the top right corner of the configuration area, there is a control bar with a red stop button, a green play button, and a refresh button, all enclosed in a blue box.

Click to Start/Stop the Batch Poller.

Set the schedule for the batch poller to look for newly submitted jobs.



Integrating Data

Continuous Loads

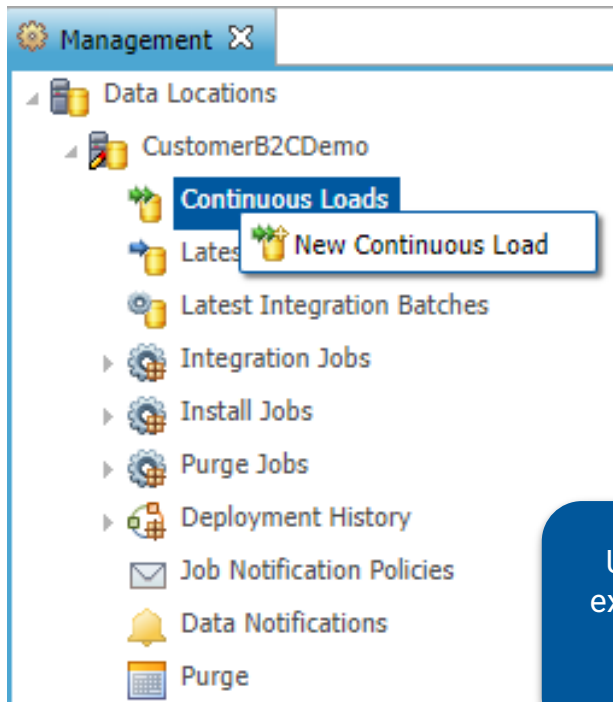
```
ace:!0,async:!1,global:!1,"throws":!0}}},n
re(this[0]),b.map(function(){var
is).wrapInner(a.call(this,b))}:this.each(f
function(){return this.parent().each(function
idden"===a.type)return!0;a=a.parentNode}ret
urn!n.expr.filters.hidden(a)};var Zb=%20/g,
"object"===typeof e&&null!=e?b:"")+"]",e,c,d)
)+"="+encodeURIComponent(b)};if(void
return d.join("&").replace(Zb,"+")),n.fn.exte
a=this.type;return this.name&&!
on(a){return{name:b.name,value:a.replace(_b,
igger -> action -> reward -> investment)$/i.
jax=!!fc,fc&&n.ajaxTransport(function(b){if(
rideMimeType&&g.overrideMimeType(b.mimeType)
lf(c&&(d||4===g.readyState))if(delete ec[h],
isLocal||b.crossDomain?1223===f&&(f=204):f=
new a.XMLHttpRequest}catch(b){})function hc
vascript, application/ecmascript, applicati
Domain&&(a.type="GET",a.global=!1)),n.ajax
b.url,b.onload=b.onreadystatechange=fu
```

Continuous Loads

- Continuous Loads enable data to be pushed into the MDM hub in a continuous way without load initialization or load submission
- With Continuous Loads:
 - No need for INTEGRATION_LOAD.GET_NEW_LOADID or SUBMIT_LOAD
 - Use a **dedicated Load ID**, and xDM **automatically submits** them for you
 - Each subsequent load made with this load ID is processed using the next interval
 - Subsequent data loads made with this load ID are processed at the next interval
- **Delayed Continuous Loads** options prevents job contention by delaying load submission in the event of long-running certification job instances.

Continuous Load - creation

- Configure Continuous Loads in Data Locations in Management view
- It provides a reusable LoadId



Create New Continuous Load

Continuous Load

❌ Program Name must have a value.

Active: ☐

Name:

Program Name:

On Submit Job:

Submit Interval:

Submit as:

Wait for Previous Loads to Finish: ☒

Finish Cancel

User who submits external loads (does not have to be in security realm)

For information purposes only

Make sure job processes all entities in load

Continuous Load - review

- Now submit records using Load ID 9394

This continuous load uses ID 9394.

Continuous Load: 9394

TestModel ▾ RootBranch ▾ 9394 ▾

Details

Data Branch: RootBranch ... ✕

Load ID: 9394

Active: ☒

Program Name: WebServicesIntegration

Description:

On Submit Job: INTEGRATE_PARTY ... ✕

Submit Interval: 10

Submit as: semadmin

Integration Load Type: Continuous Load ▾



Integrating Data

Error Management

```
ace:!0,async:!1,global:!1,"throws":!0}}},n
re(this[0]),b.map(function(){var
is).wrapInner(a.call(this,b))}:this.each(f
function(){return this.parent().each(function
idden"===a.type)return!0;a=a.parentNode}ret
urn!n.expr.filters.hidden(a)};var Zb=/%20/g,
"object"===typeof e&&null!=e?b:"")+"]",e,c,d)
)+"="+encodeURIComponent(b)};if(void
return d.join("&").replace(Zb,"+")),n.fn.exte
a=this.type;return this.name&&!
on(a){return{name:b.name,value:a.replace(_b,
igger -> action -> reward -> investment)$/i.
jax=!!fc,fc&&n.ajaxTransport(function(b){if(
rideMimeType&&g.overrideMimeType(b.mimeType)
lf(c&&(d||4===g.readyState))if(delete ec[h],
isLocal||b.crossDomain?1223===f&&(f=204):f=
new a.XMLHttpRequest}catch(b){})function hc
vascript, application/ecmascript, applicati
Domain&&(a.type="GET",a.global=!1)),n.ajaxT
b.url,b.onload=b.onreadystatechange=fu
```

Jobs – Loads are transactional

Jobs are sequential, and past jobs cannot be re-run

- A job left behind cannot be re-run
- **Transactions must run in their submission order**

Once a load is submitted, it cannot be halted

- **Submit is the equivalent of a DB commit**

Example: Suspend on error

Execution Engine Demo Applications

Execution Engine

▼ Execution Engine Activity

Queued Jobs: 0
Running Jobs: 1

▼ Execution Engine Cluster and Queues

Cluster/Queue Name	Status	Queued Jobs	Running Jobs	Suspend On Error
CustomerMDMHub	READY	0	1	☒
Q0.IntegrationQueue	SUSPENDED	0	1	☒
SEM_SYS_QUEUE	READY	0	0	☐
SEM_SYS_CLUSTER	READY	0	0	☐
SEM_SYS_QUEUE	READY	0	0	☐

Job Log: INTEGRATE_DATA

INTEGRATE_DATA ▼

Details
Job Log Parameters

► Details

▼ Tasks

Name	Status	Start Date
CostCenter	Done	Fri Sep 23 16:34:05.354 +0200
Employee	Done	Fri Sep 23 16:34:07.912 +0200
Customer	Error	Fri Sep 23 16:34:07.912 +0200

Task Group Log: Customer

INTEGRATE_DATA ▼ Customer ▼

Details

▼ Details

Name: Customer

Task Definition: Customer

Parent Group Task Log:

Job Log: INTEGRATE_DATA

Status: Error

Start Date: 23.09.2011 16:34:05.354 +0200

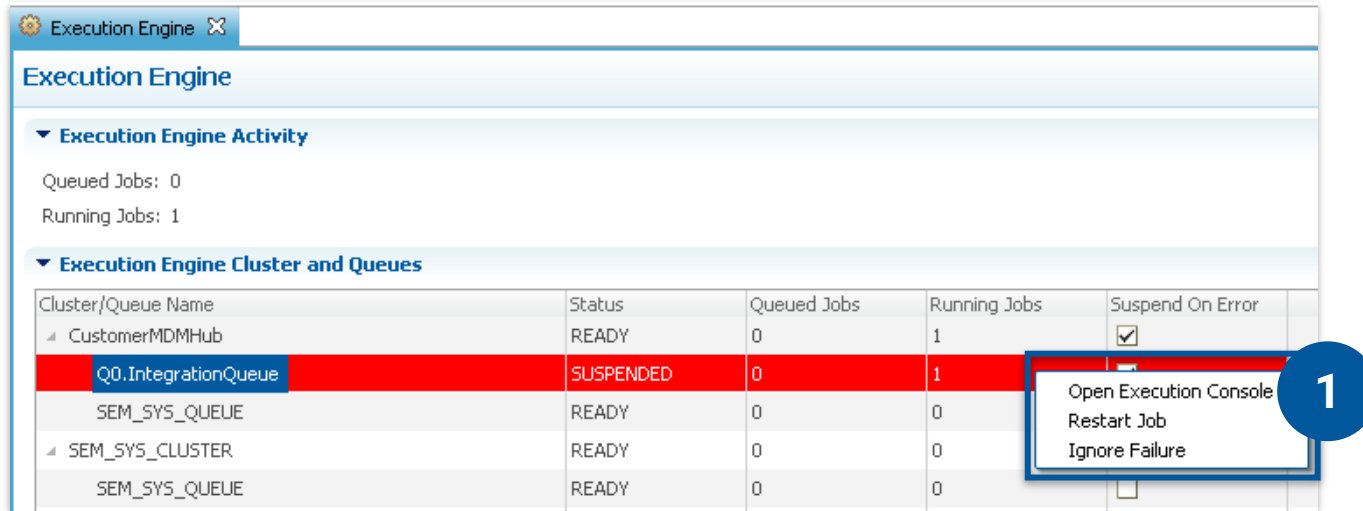
End Date: 23.09.2011 16:34:07.912 +0200

Duration: 0 00:00:02 558,000000

Message:

Une exception SQL a été capturée lors de l'exécution de l'opération
+ ORA-00942: Table ou vue inexistante

Example: Restarting a job



Execution Engine

Execution Engine Activity

Queued Jobs: 0
Running Jobs: 1

Execution Engine Cluster and Queues

Cluster/Queue Name	Status	Queued Jobs	Running Jobs	Suspend On Error
CustomerMDMHub	READY	0	1	☒
Q0.IntegrationQueue	SUSPENDED	0	1	☒
SEM_SYS_QUEUE	READY	0	0	☐
SEM_SYS_CLUSTER	READY	0	0	☐
SEM_SYS_QUEUE	READY	0	0	☐

1

- Open Execution Console
- Restart Job
- Ignore Failure

Cluster/Queue Name	Status	Queued Jobs	Running Jobs	Suspend On Error
CustomerMDMHub	READY	0	0	☒
Q0.IntegrationQueue	READY	0	0	☒
SEM_SYS_QUEUE	READY	0	0	☐
SEM_SYS_CLUSTER	READY	0	0	☐
SEM_SYS_QUEUE	READY	0	0	☐

Job appears as running
then disappears when it is
complete.



Integrating Data

Logs

```
ace:!0,async:!1,global:!1,"throws":!0}}),n
re(this[0]),b.map(function(){var
is).wrapInner(a.call(this,b))}:this.each(f
function(){return this.parent().each(function
idden"===a.type)return!0;a=a.parentNode}retu
urn!n.expr.filters.hidden(a)};var Zb=%20/g,
"object"===typeof e&&null!=e?b:"")+"]",e,c,d)
)+"="+encodeURIComponent(b)};if(void
return d.join("&").replace(Zb,"+")),n.fn.exte
a=this.type;return this.name&&!
on(a){return{name:b.name,value:a.replace(_b,
igger -> action -> reward -> investment)$/i.
jax=!!fc,fc&&n.ajaxTransport(function(b){if(
rideMimeType&&g.overrideMimeType(b.mimeType)
lf(c&&(d||4===g.readyState))if(delete ec[h],
isLocal||b.crossDomain?1223===f&&(f=204):f=
new a.XMLHttpRequest}catch(b){})function hc
vascript, application/ecmascript, applicati
Domain&&(a.type="GET",a.global=!1)),n.ajaxT
b.url,b.onload=b.onreadystatechange=fu
```

Job Logs Editor

Toolbar: filtering, automatic width size,
purging or deleting job logs

Semarchy xDM | Application Builder

Management **Executions**

- Data Locations
 - IntelligentDataHub
- Models Administration
 - BusinessGlossary
 - IntelligentDataHub
- Job Executions
 - Execution Engine
 - Integration Batch Poller
 - Executions**

Job Logs

Displaying 100 out of 188 items

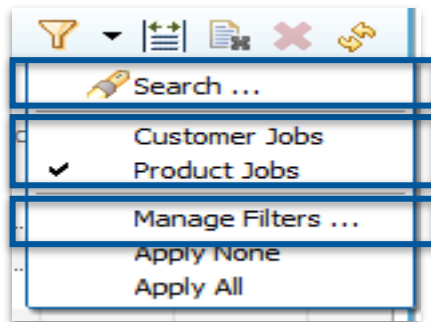
	Name	Start D...	End Date	Duration	Row C...	Insert ...	Update...	Merge ...	Delete ...	Select ...	Error c...	Message	Create...
☒	DAJ_AuthorSectorX...	25/03/...	25/03/...	0 year...	3	1	0	1	0		0		BatchP...
☒	B2B	25/03/...	25/03/...	0 year...	6209	958	2726	565	0	1089	0		BatchP...
☒	DB Patch 3/25/20 4...	25/03/...	25/03/...	0 year...									matt.d...
☒	B2B	25/03/...	25/03/...	0 year...	6563	958	2726	565	0	1089	0		BatchP...
☒	ADDRESS_CACHE	25/03/...	25/03/...	0 year...	913		547	183		183	0		BatchP...
☒	DB Patch 3/25/20 2...	25/03/...	25/03/...	0 year...									matt.d...
☒	B2B	24/03/...	24/03/...	0 year...	2	0	0	0	0	0	0		BatchP...
☒	B2B	23/03/...	23/03/...	0 year...	907	179	191	73	3	45	0		BatchP...
☒	B2B	23/03/...	23/03/...	0 year...	8847	1711	2768	871	0	1089	0		BatchP...
☒	B2B	23/03/...	23/03/...	0 year...	231	126	19	32	0	0	0		BatchP...
☒	B2B	23/03/...	23/03/...	0 year...	6529	958	2719	565	0	1089	0		BatchP...
☒	PURGE_DATA_BRA...	22/03/...	22/03/...	0 year...	0				0				PurgeS...
☒	PURGE_DATA_BRA...	15/03/...	15/03/...	0 year...	2				2				PurgeS...
☒	PURGE_DATA_BRA...	08/03/...	08/03/...	0 year...	13				13				PurgeS...
☒	DAJ_EditIndustryA...	04/03/...	04/03/...	0 year...	48	4	4	20	0		0		BatchP...

Defining Job Log Filters

Create a quick filter.

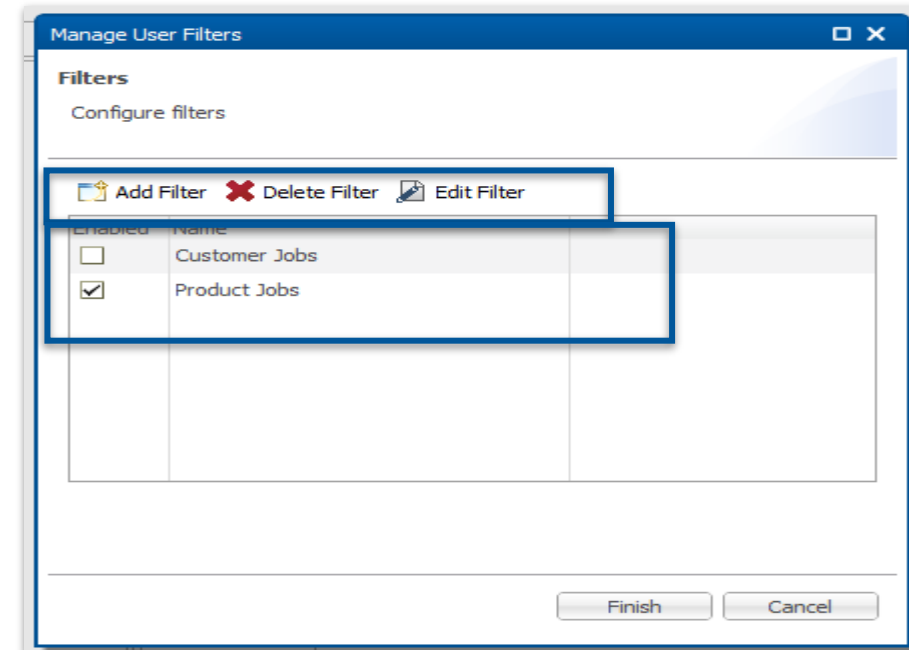
Existing Filters for the Job Logs view. One of them is currently active.

Manage Job Log Filters



Add/Remove/Edit existing Filters.

Existing Job Logs Filters



Job Log Filter - example

Jobs named like '%PRO%'

Filter the period for the last 30 days

Optionally Save your Filter

The screenshot shows a 'Define Filter' dialog box with the following fields and options:

- Job Name:** %PRO%
- Created By:** (empty text box)
- Status:** DONE,ERROR,WARNING (with a dropdown arrow)
- ☒ Only Include
- ☒ After ☐ Before
- Last:** 30 **Days** (with a dropdown arrow)
- ☒ Save as Preferred Filter
- Filter Name:** Product Jobs
- Buttons:** Finish, Cancel

Purging Job Logs (using manual selection)

The screenshot shows the 'Job Logs' window with a table of logs. A blue selection box highlights four rows. A red 'X' icon in the toolbar is circled with a '2'. A confirmation dialog box is open in the foreground, asking 'Are you sure you want to definitely remove these objects?'. The 'OK' button in the dialog is circled with a '3'. A text label 'Select the Job Logs you want to remove' points to the selected rows, which are also indicated by a '1' in a blue circle.

Job Logs

Displaying 6 out of 6 filtered items

	Name	Start Date	End Date	Duration	Row Count	Insert Count	Update Count	Merge Count	Delete Count	Select
☒	INTEGRATE_PRODU	2011-09-09 12:25	2011-09-09 12	0 0:0:17.	11162	54	0	5554	0	
☒	INTEGRATE_PRODU	2011-09-09 12:14	2011-09-09 12	0 0:0:18.	254	0	0	24	0	
☒	INTEGRATE_PRODU	2011-09-09 12:13	2011-09-09 12	0 0:0:18.	278	0	0	24	0	
☒	INTEGRATE_PRODU	2011-09-09 12:13	2011-09-09 12	0 0:0:17.	470	24	0	72	0	
☒	INTEGRATE_PRODU	2011-09-09 12:13	2011-09-09 12	0 0:0:19.	18008	0	0	8916	0	
☒	INTEGRATE_PRODU	2011-09-09 12:07	2011-09-09 12	0 0:0:17.	11236	36	0	3366	0	

Job Logs

Are you sure you want to definitely remove these objects?

OK Cancel

Select the Job Logs you want to remove

Purging Job Logs (using filters)

The screenshot shows the 'Job Logs' window with a table of logs. A blue box labeled '1' highlights the 'Purge' icon in the top right toolbar. Overlaid on this is the 'Purge Logs' dialog box. A blue box labeled '2' highlights the 'Purge Log Filter' section, which includes the 'Use Existing Filter' radio button, a dropdown menu showing 'Product Jobs', and the 'Define New Filter' radio button. A blue box labeled '3' highlights the 'Finish' button at the bottom right of the dialog box.

Job Logs

Displaying 19 out of 19 filtered items

	Name	Start Date	End Date	Duration
☒	INTEGRATE_OFFER:	2011-09-26 18:32	2011-09-26 18:32	0 0:0:0
☒	DBSchema Install Cu	2011-09-26 09:53	2011-09-26 09:53	0 0:0:0

Purge Logs

Purge Job Logs using a Filter

☒ Use Existing Filter:

Filter: Product Jobs

☐ Define New Filter:

< Back Next > Finish Cancel

Select an existing filter that will be used to determine which job logs to purge

... or define a new filter for the purge